

# YMÜ227 Nesne Tabanlı Programlama

*Metin Tabanlı Mini–Macera Oyunu*

Öğretim Elemanı: Doç.Dr.Sinem AKYOL

Öğrenci Adı Soyadı: Merve KARAGÜLLE

Öğrenci Numarası:2402290087

Proje Teslim Tarihi: 28.11.2025

# İçindekiler

1. Giriş
2. Proje Amacı
3. Geliştirme Süreci
4. Sistem Tasarımı ve Sınıf Yapısı
  - 4.1 Oda (Oda, TuzakOdasi, IyilesmeOdasi)
  - 4.2 Eşya (Eşya, AnahtarEşya, IksirEşya, SilahEşya)
  - 4.3 Oyuncu (Oyuncu)
  - 4.4 NPC (NPC, DostNPC, DusmanNPC)
  - 4.5 Konuşma Yapısı (KonusmaDugumu, KonusmaSecenegi)
  - 4.6 Oyun Motoru (OyunMotoru)
  - 4.7 Main Sınıfı (Main)
5. Oyun Akışı
  - 5.1 Başlatma (INIT) Aşaması
  - 5.2 Oyun Döngüsü (RUNNING) Aşaması
  - 5.3 Bitiş (END) Aşaması
6. Girdi / Çıktı Örnekleri
7. Hata ve İstisna Yönetimi
  - 7.1 Uygulanan Hata Kontrolleri ve Mesajları
  - 7.2 Sistemsel İstisna Yönetimi
  - 7.3 Bu Tasarımın Sağladığı Avantajlar
8. Kaynak Kodlar Hakkında Açıklamalar
  - 8.1 Dosya Yapısının Genel Düzeni
  - 8.2 Kodun Genel Yazım Kalitesi
9. Test ve Senaryolar
10. Sonuç ve Değerlendirme
- Genel Değerlendirme Tablosu

## 1. Giriş

Bu proje kapsamında Java programlama dili kullanılarak konsol tabanlı bir “**Metin Tabanlı Mini-Macera Oyunu**” geliştirilmiştir. Oyuncu, birbirine bağlı odalardan oluşan bir oyun dünyasında dolaşmakta, odalar arasında geçiş yapmakta, eşyalar toplamakta, envanterindeki eşyaları kullanmakta ve karşılaştığı karakterlerle (NPC) etkileşime geçebilmektedir.

Oyun tamamen metin tabanlıdır ve yönetim, klavyeden komut girilerek yapılmaktadır. Oyuncu, örneğin “**bak**”, “**git kuzey**”, “**al kırmızı\_anahtar**”, “**kullan kırmızı\_anahtar**”, “**konus Muhafız**”, “**env**”, “**durum**” gibi komutlarla oyun dünyasında ilerler. Her komut, arka planda bir veya birden fazla sınıfın iş birliği ile çalışmaktadır.

Bu yapı sayesinde nesne tabanlı programlamanın temel kavramları olan sınıf, nesne, kalıtım, polimorfizm, kompozisyon ve hata yönetimi gibi başlıklar pratik olarak uygulanmıştır.

## 2. Proje Amacı

Bu projenin temel amacı, Java programlama diliyle nesne tabanlı programlama (OOP) yaklaşımını gerçek bir senaryo üzerinden uygulamalı olarak öğrenmek ve pekiştirmektir. Proje boyunca bir oyun kurgusunu sıfırdan tasarlayarak, farklı sınıfların birbiriyle nasıl etkileşime girdiğini, kalıtım ilişkilerinin nasıl kurulduğunu ve polimorfizmin gerçek bir uygulamada nasıl kullanıldığını deneyimleme fırsatı elde ettim.

Bu kapsamda proje bana özellikle şu konularda beceri kazandırmayı hedeflemektedir:

- **Kalıtım (Inheritance) ve Polimorfizm (Polymorphism):**  
Bir üst sınıf üzerinden ortak davranışları tanımlayıp, alt sınıfların bu davranışları kendine özgü biçimde genişletmesini sağladım. Örneğin, *Esys* sınıfı soyut bir temel yapı, *AnahtarEsys*, *IksirEsys* ve *SilahEsys* ise farklı kullanım davranışları olan alt sınıflar olarak tasarlandı.
- **Soyut Sınıflar ve Override Mekanizması:**  
Soyut metotlar üzerinden genel bir kalıp oluşturup, bu metotları alt sınıflarda yeniden tanımlayarak (override) her eşyanın farklı bir etkileşim sağlamasını gerçekleştirdim. Bu yaklaşım sayesinde polimorfik davranışı doğrudan uygulamış oldum.
- **Koleksiyon Yapılarıyla Nesneler Arası İlişkilerin Yönetilmesi:**  
Odaların çıkış bağlantıları *Map* ile, odadaki eşyalar ve NPC'ler ise *List* yapılarıyla yönetildi. Bu sayede dinamik olarak oda yapıları oluşturmayı ve nesneler arası ilişkileri koleksiyonlar üzerinden kontrol etmeyi öğrendim.

- **Konsol Tabanlı Oyun Döngüsü Uygulamak:**  
Kullanıcıdan sürekli komut alıp işleyen bir döngü tasarlayarak, komut yorumlama, hata ayıklama, yönlendirme ve oyun akışını kontrol etme gibi konularda pratik kazandım.
- **Hata Yönetimi (Exception Handling):**  
Kullanıcının hatalı girişlerinde oyunun durmaması için try-catch yapıları ile güvenli bir akış oluşturdum. Ayrıca "bilinmeyen komut", "bu eşya sende yok", "bu yönde çıkış yok" gibi kullanıcıya açıklayıcı hata mesajları vererek doğru geri bildirim sağlamayı öğrendim.

Bu proje sayesinde yalnızca teorik olarak öğrendiğim OOP kavramlarını pratiğe dökmedim; aynı zamanda nesne tabanlı bir mimariyi baştan sona nasıl tasarlayacağımı ve yönetebileceğimi de deneyimlemiş oldum.

### 3. Geliştirme Süreci

Bu proje, planlı ve aşamalı bir yaklaşımla üç temel adım üzerinden geliştirilmiştir. Her aşamada hem teorik bilgilerden hem de uygulamalı denemelerden yararlanarak oyunun hem sağlam bir altyapıya sahip olması hem de kullanıcı açısından anlaşılır bir yapıda ilerlemesi hedeflenmiştir.

---

#### 1) Tasarım Aşaması

Geliştirme sürecine başlamadan önce oyunun genel yapısı kağıt üzerinde planlanmıştır. Bu aşamada:

- **Oyun dünyasının haritası** çıkarılmış, en az beş odadan oluşan bir yapı oluşturulmuştur.  
Her odanın hangi odaya hangi yönden bağlanacağı belirlenmiş, böylece oyun içinde tutarlı bir mekânsal düzen sağlanmıştır.
- Her odada **hangi eşyaların (item)** ve **hangi NPC'lerin** bulunacağı belirlenmiştir. Oyuncunun ilerlemesi için gerekli olan kırmızı anahtar, iksir ve silah gibi eşyaların konumları dikkatli bir şekilde dağıtılmıştır.
- Oyunda kullanılacak sınıflar ve aralarındaki ilişkiler taslak hâlinde çizilmiştir. Örneğin:
  - *Esys* sınıfından türeyen *AnahtarEsys*, *IksirEsys*, *SilahEsys*
  - *NPC* sınıfından türeyen *DostNPC* ve *DusmanNPC*
  - Oda yapısının genişletilmesi amacıyla *TuzakOdasi* ve *IyilesmeOdasi* gibi alt sınıflar öngörülmüştür.

- Oyuncunun komutlarla etkileşimini yönetecek bir **OyunMotoru (Game Engine)** sınıfının görevleri belirlenmiştir.

Bu aşama, projenin geri kalanının sorunsuz ilerlemesi için temel bir yol haritası oluşturmuştur.

---

## 2) Uygulama (Kodlama) Aşaması

Tasarım tamamlandıktan sonra kodlama aşamasına geçilmiştir. Bu aşamada yapı, adım adım genişletilerek oyunun temel fonksiyonları eklenmiştir.

- İlk olarak projenin çekirdeğini oluşturan temel sınıflar oluşturulmuştur:  
**Oda, Oyuncu, Esys, NPC, OyunMotoru ve Main.**
- Ardından nesne tabanlı programlamanın gerektirdiği şekilde **kalıtım kullanılan alt sınıflar** yazılmıştır:
  - Oda'dan türeyen: **TuzakOdasi, IyilesmeOdasi**
  - Esys'dan türeyen: **AnahtarEsys, IksirEsys, SilahEsys**
  - NPC'den türeyen: **DostNPC ve DusmanNPC**
- Dost NPC'lerle konuşma sisteminin çalışması için bir **konuşma ağacı (dialog tree)** yapılandırılmıştır.  
Bunun için iki sınıf tanımlanmıştır:
  - **KonusmaDugumu:** Her konuşma adımının metnini tutar.
  - **KonusmaSecenegi:** Oyuncunun seçebileceği seçenekleri temsil eder.

Bu aşama boyunca sınıflar arasında kompozisyon ilişkileri kurularak odalar, eşyalar ve karakterler birbirine entegre edilmiştir. Oyunun akışını yöneten OyunMotoru sınıfında komutların işlenmesi, hataların yakalanması ve oyun döngüsünün devam etmesi gibi temel mekanizmalar uygulanmıştır.

---

## 3) Test ve Düzeltme Aşaması

Kodlama tamamlandıktan sonra oyun, farklı senaryolar altında test edilmiştir. Bu aşama oyunun kararlılığını ve kullanıcı dostu olmasını sağlamak açısından büyük önem taşımıştır.

- Oyuncunun kullanabileceği tüm komutlar farklı kombinasyonlarla test edilmiştir: **bak, git, al, kullan, konus, sec, env, durum, yardım, cikis**
- **Geçersiz komutlar, olmayan eşya veya NPC, kilitli kapı, yanlış yön girişleri** gibi durumlarda oyunun doğru ve açıklayıcı hata mesajları verdiği doğrulanmıştır.
- Düşman NPC'lerle karşılaşmalarda saldırı mekanizması test edilmiş, oyuncunun canının azaldığı ve düşmanın yenildiğinde odadan silindiği görülmüştür.
- Özel oda türlerinin (TuzakOdasi ve IyilesmeOdasi) ilk girişte tetiklenen davranışları kontrol edilmiştir.
- Kullanıcıdan kaynaklı hatalarda oyun kapanmasın diye **try-catch bloklarıyla istisna yönetimi** uygulanmış ve test edilmiştir.

Bu testler sonucunda eksiklikler giderilmiş, komut yorumlama sistemi güçlendirilmiş ve oyunun daha akıcı çalışması sağlanmıştır.

#### 4. Sistem Tasarımı ve Sınıf Yapısı

Bu projede metin tabanlı bir mini macera oyunu geliştirmek için nesne tabanlı programlama (OOP) prensipleri doğrultusunda bir sınıf yapısı tasarlanmıştır. Tüm sınıflar tek bir Java dosyasında yer almakla birlikte, mantıksal olarak rollerine göre ayrılmıştır. Bu bölümde, projede yer alan tüm sınıflar alt başlıklar hâlinde detaylı bir şekilde incelenmiştir.

---

##### 4.1 Oda Sistemi (Oda, TuzakOdasi, IyilesmeOdasi)

###### Oda Sınıfı

Oyun dünyasındaki tüm mekânları temsil eder. Her oda bir konumdur ve oyuncu bu odalar arasında gezinir.

###### Temel özellikleri:

- id, ad, acıklama gibi açıklayıcı bilgiler içerir.
- Diğer odalara yön bağlantıları Map<String, Oda> ile saklanır.
- Oda içerisindeki eşyalar List<Esys> ile tutulur.
- Odada bulunan NPC'ler List<NPC> yapısında saklanır.
- odaGirildi() metodu her oda girişinde tetiklenir (özelleştirme için kullanılır).

Bu sınıf; hem oyun dünyasını oluşturmak hem de diğer sınıfların ilişkilendirileceği temel yapıyı sağlamak açısından merkezi bir role sahiptir.

---

### **TuzakOdasi Sınıfı**

Oda sınıfından kalıtım alır.

#### **Temel amacı:**

- Oyuncu bu odaya ilk kez girdiğinde otomatik olarak hasar almasını sağlar.
- odaGirildi() metodu override edilerek tuzak davranışı eklenmiştir.

Bu sınıf kalıtım ve polimorfizmin oyun mekaniğine uygulanışına iyi bir örnektir.

---

### **İyileşmeOdasi Sınıfı**

Oda sınıfının başka bir alt sınıfıdır.

#### **Özellikleri:**

- Oyuncu odaya ilk girişinde can puanı artırılır.
- İyileştirme işlemi yalnızca bir kez uygulanır.

Bu sınıf, genişletilmiş oda davranışlarının nasıl tasarlanabileceğini göstermektedir.

---

## **4.2 Eşya Sistemi (Eşya, AnahtarEşya, IksirEşya, SilahEşya)**

### **Eşya Sınıfı (Soyut Sınıf)**

Tüm eşya türlerinin ortak özelliklerini tanımlayan abstract bir sınıftır.

#### **Ortak özellikler:**

- id
- komutAdi
- aciklama

#### **Soyut metot:**

- kullan(Oyuncu, OyunMotoru)  
→ Her alt sınıf bu metodu kendi işlevine göre override eder.
- 

### **AnahtarEsysa Sınıfı**

Anahtar, oyuncunun kilitli bir kapıyı açmasını sağlar.

#### **Özellikleri:**

- Hangi yönün kilidini açacağı belirtilir.
  - Kullanıldığında ilgili yön açılır ve oyuncu yeni odaya geçer.
- 

### **IksirEsysa Sınıfı**

Oyuncunun canını artıran tek kullanımlık eşyadır.

#### **Özellikleri:**

- Kullanıldığında belli bir miktar can kazandırır.
  - Kullanımdan sonra envanterden silinir.
- 

### **SilahEsysa Sınıfı**

Oyuncunun saldırı gücünü artıran bir eşyadır.

#### **Özellikleri:**

- Kalıcı saldırı gücü artışı sağlar.
  - Silahlar kaybolmaz, oyuncu üzerinde kalıcı etki bırakır.
- 

## **4.3 Oyuncu Sınıfı (Oyuncu)**

Oyuncu, oyun boyunca kontrol edilen karakterdir.

#### **Temel özellikleri:**

- Bulunduğu oda



- Envanter (List<Esys>)
- Can puanı (varsayılan 100)
- Saldırı gücü (varsayılan 5)

#### **Temel metotlar:**

- hareketEt() → odalar arasında geçiş
- esyaAl() → eşyayı envantere ekleme
- esyaKullan() → polimorfik kullanım davranışını çalıştırır
- hasarAl(), canArtir(), saldiriGucuArtir() → istatistik kontrolü
- durumYaz(), envanteriYaz() → oyuncu bilgilerini gösterir

Bu sınıf oyunun aktif olarak işleyen en önemli nesnesidir.

---

### **4.4 NPC Sistemi (NPC, DostNPC, DusmanNPC)**

#### **NPC (Üst Sınıf)**

Tüm karakterlerin ortak üst sınıfıdır.

#### **Sahip olduğu tek özellik:**

- ad

Bu yapı basit görünse de kalıtım için temel oluşturur.

---

#### **DostNPC Sınıfı**

Oyuncu ile etkileşimli konuşmalar yapabilen karakterlerdir.

#### **Özellikleri:**

- Her DostNPC'nin bir konuşma ağacı kök düğümü vardır.
  - Oyuncu "konuş" komutu ile diyalog başlatabilir.
-

## **DusmanNPC Sınıfı**

Oyuncuya saldırabilen düşman karakterlerdir.

### **Özellikleri:**

- can ve saldiriHasari gibi savaş mekanikleri içerir.
  - Oyuncuya hasar verir.
  - Canı bittiğinde odadan kaldırılır.
- 

## **4.5 Konuşma Yapısı (KonusmaDugumu, KonusmaSecenegi)**

Oyun içindeki dost NPC'lerle gerçek bir konuşma deneyimi oluşturmak amacıyla bir konuşma ağacı sistemi tasarlanmıştır.

### **KonusmaDugumu**

- Her düğüm konuşma metni içerir.
- Birden fazla seçenek (KonusmaSecenegi) barındırabilir.

### **KonusmaSecenegi**

- Oyuncunun seçebileceği cevapları temsil eder.
- Her seçenek bir sonraki KonusmaDugumu'na bağlanabilir.

Bu sistem, çok adımlı ve dallanabilen NPC diyalogları oluşturmayı mümkün kılar.

---

## **4.6 Oyun Motoru (OyunMotoru)**

OyunMotoru sınıfı oyunun en merkezi sınıfıdır ve oyun döngüsünün tamamını kontrol eder.

### **Görevleri:**

- Odaları, eşyaları ve NPC'leri oluşturur.
- Kullanıcıdan gelen tüm komutları işler.
- Kilitli kapıları yönetir.

- Konuşma sistemini yürütür.
- Hatalı komutları yakalar, yönlendirir.
- Oyun döngüsünü başlatır ve yönetir.

#### **Komut sistemi tamamen Türkçedir:**

- bak
- git <yön>
- al <eşya>
- kullan <eşya>
- konus <npc>
- sec <n>
- env / envanter
- durum
- yardım
- cikis

Bu sınıf, oyunun beynidir ve diğer tüm sınıflarla aktif olarak iletişim kurar.

---

#### **4.7 Main Sınıfı (Main)**

En basit fakat en gerekli sınıftır.

##### **Görevi:**

OyunMotoru'nun bir örneğini oluşturarak oyunu başlatmak.

```
public static void main(String[] args) {  
    OyunMotoru motor = new OyunMotoru();  
    motor.baslat();  
}
```

Bu sınıf, uygulamanın giriş noktasıdır ve oyunun çalışmasını sağlar.

## 5. Oyun Akışı

Oyun akışı, OyunMotoru sınıfının baslat() metodunun çalıştırılmasıyla başlar. Bu metot oyunun başlangıç hazırlıklarını yapar, kullanıcı ile etkileşimi başlatır ve oyun döngüsünü yönetir. Oyun, üç ana aşamadan oluşan bir süreç içinde çalışır: **başlatma (INIT)**, **oyun döngüsü (RUNNING)** ve **bitiş (END)**.

---

### 5.1 Başlatma (INIT) Aşaması

Oyunun başlatılabilmesi için dünya elemanlarının oluşturulması gerekir. Bunun için dunyayiHazirla() metodu çağrılır.

Bu aşamada:

#### • Odalar oluşturulur

En az beş adet oda tanımlanmış ve yön bağlantıları şu şekilde belirlenmiştir:

- Salon
- Koridor
- Silah Odası
- Büyük Salon (İyileştirme Odası)
- Zindan (Tuzak Odası)

Her oda, bagla() metodu ile kuzey–güney–doğu–batı yönlerinde birbirine bağlanmıştır. Böylece oyuncunun gezinebileceği mekânsal bir yapı oluşturulmuştur.

#### • Eşyalar oyuna yerleştirilir

Her oda, çeşitli eşya nesneleri ile donatılmıştır:

- Kırmızı Anahtar
- Küçük İksir
- Demir Kılıç

Bu eşyalar, soyut Esya sınıfından türetilen alt sınıflardır ve her biri farklı kullanım davranışına sahiptir.

#### • NPC'ler konumlandırılır

Salon’da bir dost NPC (Muhafız),  
Zindan’da ise bir düşman NPC (Goblin) bulunur.

#### • Oyuncu başlangıç odasına yerleştirilir

Oyuncu, oyun başladığında “Salon” içerisinde konumlandırılır ve durumu ekrana yazılır.

---

## 5.2 Oyun Döngüsü (RUNNING) Aşaması

Başlatma tamamlandıktan sonra oyun sürekli bir döngü içinde ilerler. Bu döngü, kullanıcıdan her komut girişinde oyunun yeniden güncellenmesini sağlar.

Oyun döngüsü şu adımlardan oluşur:

---

### 1) Komut İpuçlarının Gösterilmesi

Oyun, her turda oyuncuya örnek olabilecek bazı komutları listeler:

- Odada eşya varsa: *al <esya>*
- NPC varsa: *konus <npc>*
- Genel komutlar: *bak, git kuzey, env, durum*

Bu sayede oyuncuya yönlendirme sağlanır.

---

### 2) Kullanıcıdan Komut Alınması

Kullanıcıdan bir metin girdisi istenir:

- Boş komut girilirse uyarı verilir.
  - Komut metni boşluklara göre parçalanır.
  - İlk kelime **esas komut**, geri kalan kelimeler **parametre** olarak değerlendirilir.
- 

### 3) Komutun İşlenmesi

Komutlar switch yapısı ile değerlendirilir ve ilgili işlemler gerçekleştirilir.

**Desteklenen komutlar:**

---

### ❖ 1) bak

Bulunulan odanın:

- açıklamasını,
- çıkış yönlerini,
- içindeki eşyaları,
- NPC'leri  
ekrana yazar.

---

### ❖ 2) git <yon>

Oyuncunun odadan bir başka odaya geçmesini sağlar.

Kullanılabilir yönler:

- kuzey (k)
- guney (g)
- dogu (d)
- bati (b)

Kontroller:

- O yönde çıkış yoksa uyarı verilir.
- Çıkış kilitli ise “Kırmızı kapı kilitli” mesajı gösterilir.
- Çıkış açıksa oyuncu yeni odaya taşınır ve oda özellikleri tetiklenir (tuzak, iyileştirme vb.)

---

### ❖ 3) al <esya\_kodu>

Bulunulan odadaki bir eşyayı envantere ekler.

Kontroller:

- Eşya yoksa hata mesajı verilir.

- Eşya başarıyla alınırsa odadan kaldırılır.
- 

#### ❖ 4) **kullan** <esya\_kodu>

Envanterde bulunan bir eşyanın kullanım davranışını çalıştırır.

Her eşya kendi kullanım fonksiyonunu override ettiği için polimorfik bir yapı sağlanmıştır.

Örnek işlevler:

- Anahtar: kilitli kapıyı açar.
  - İksir: can artırır.
  - Silah: saldırı gücünü yükseltir.
- 

#### ❖ 5) **konus** <npc\_ad>

Dost NPC ile konuşmayı başlatır.

Yapılan işlemler:

- NPC dost ise konuşma ağacı gösterilir.
  - NPC düşman ise konuşma yerine saldırı gerçekleşir.
- 

#### ❖ 6) **sec** <numara>

Konuşma yapısında bir seçenek seçmeyi sağlar.

- Yanıt seçilir, diyalog bir sonraki düğüme geçer.
  - Son düğüme gelinirse konuşma sona erer.
- 

#### ❖ 7) **env** / **envanter**

Oyuncunun envanterindeki tüm eşyalar listelenir.

---

#### ❖ 8) durum

Oyuncunun temel istatistikleri gösterilir:

- Can
  - Saldırı gücü
  - Bulunduğu oda
- 

#### ❖ 9) yardım

Tüm komutların açıklamalarını ekrana yazdırır.

---

#### ❖ 10) cikis

Oyun döngüsünü sonlandırır ve oyun kapatılır.

---

### 5.3 Bitiş (END) Aşaması

- Oyuncu isteyerek “cikis” komutunu girerse veya
- Canı 0'a düşerse

oyun sona erer.

Oyuncuya teşekkür mesajı yazılır ve uygulama sonlanır.



## 6. Girdi / Çıktı Örnekleri

Bu bölümde oyunun çalışması sırasında konsolda görülebilecek örnek giriş ve çıkışlar yer almaktadır.

Örnek Oturum 1 – Anahtar Alma ve Kapı Açma:

```
=== Mini Macera ===
Bu bir metin tabanlı mini macera oyunudur.
Yardım için istediğin zaman 'yardim' yazabilirsin.
Örnek komutlar: 'bak', 'git kuzey', 'al kirmizi_anahtar', 'konus Muhafız', 'env', 'durum'.
Şimdi bir komut yazmayı deneyebilirsin.

Şu anda bulunduğun yer: Salon
Eski bir şatonun giriş salonundasın. Duvarlarda solmuş tablolar var.
Çıkışlar: dogu, guney, kuzey
Eşyalar: kirmizi_anahtar
Karakterler: Muhafız
=== Oyuncu Durumu ===
Bulunduğun oda : Salon
Can           : 100
Saldırı gücü  : 5

Örnek komutlar: bak | git kuzey | git guney | env | durum | al kirmizi_anahtar | konu Muhafız
Komut bekleniyor (yardim için 'yardim'): al kirmizi_anahtar
kirmizi_anahtar envanterine eklendi.

Örnek komutlar: bak | git kuzey | git guney | env | durum | konu Muhafız
Komut bekleniyor (yardim için 'yardim'): git dogu
Kırmızı kapı kilitli.

Örnek komutlar: bak | git kuzey | git guney | env | durum | konu Muhafız
Komut bekleniyor (yardim için 'yardim'): kullan kirmizi_anahtar
Kapının kilidini açtın ve dogu yönüne, Silah Odası odasına geçtin.
```

Örnek Oturum 2 – Muhafız ile Konuşma:

```
=== Mini Macera ===
Bu bir metin tabanlı mini macera oyunudur.
Yardım için istediğin zaman 'yardim' yazabilirsin.
Örnek komutlar: 'bak', 'git kuzey', 'al kirmizi_anahtar', 'konus Muhafız', 'env', 'durum'.
Şimdi bir komut yazmayı deneyebilirsin.

Şu anda bulunduğun yer: Salon
Eski bir şatonun giriş salonundasın. Duvarlarda solmuş tablolar var.
Çıkışlar: dogu, guney, kuzey
Eşyalar: kirmizi_anahtar
Karakterler: Muhafız
=== Oyuncu Durumu ===
Bulunduğun oda : Salon
Can           : 100
Saldırı gücü  : 5

Örnek komutlar: bak | git kuzey | git guney | env | durum | al kirmizi_anahtar | konu Muhafız
Komut bekleniyor (yardim için 'yardim'): konu Muhafız
Muhafız: "Dur! Bu şatoda ne için var?"
Seçenekler:
  1) Sakin ol, sadece çıkış yolunu arıyorum.
  2) Sen kimsin bana soru soruyorsun?
Seçim yapmak için: sec <numara> (örn: sec 1)

Örnek komutlar: bak | git kuzey | git guney | env | durum | al kirmizi_anahtar | konu Muhafız
Komut bekleniyor (yardim için 'yardim'): sec 1
Sen: Sakin ol, sadece çıkış yolunu arıyorum.
Muhafız: "Hmm... Şatoda kaybolduğunu söylüyorsun. Sana yardım edebilirim."
Seçenekler:
  1) Yardımına ihtiyacım var, ne yapmalıyım?
  2) Boşver, kendi yolumu bulurum.
Seçim yapmak için: sec <numara> (örn: sec 1)
```

### Örnek Oturum 3 – Zindanda Tuzak:

```
=== Mini Macera ===
Bu bir metin tabanlı mini macera oyunudur.
Yardım için istediğin zaman 'yardim' yazabilirsin.
Örnek komutlar: 'bak', 'git kuzey', 'al kırmızı_anahtar', 'konuş Muhafız', 'env', 'durum'.
Şimdi bir komut yazmayı deneyebilirsin.

Şu anda bulunduğun yer: Salon
Eski bir şatonun giriş salonundasın. Duvarlarda solmuş tablolar var.
Çıkışlar: dogu, guney, kuzey
Eşyalar: kırmızı_anahtar
Karakterler: Muhafız
=== Oyuncu Durumu ===
Bulunduğun oda : Salon
Can : 100
Saldırı gücü : 5

Örnek komutlar: bak | git kuzey | git guney | env | durum | al kırmızı_anahtar | konuş Muhafız
Komut bekleniyor (yardım için 'yardim'): git guney
Tuzak tetiklendi! Etrafına sivri oklar fırlıyor.
Hasar aldın! (15) Mevcut can: 85

Şu anda bulunduğun yer: Zindan
Nemli ve karanlık bir zindan. Zincir sesleri duyuyorsun...
Çıkışlar: kuzey
Eşyalar: küçük_iksir
Karakterler: Goblin
```

## 7.Hata ve İstisna Yönetimi

Oyun geliştirilirken kullanıcıdan gelebilecek hatalı girişler ile çalışma zamanında ortaya çıkabilecek istisnaların oyunu durdurmadan yönetilmesi hedeflenmiştir. Bu amaçla hem komut işleme düzeyinde hem de oyun döngüsü içinde çeşitli kontrol mekanizmaları uygulanmıştır. Kullanıcının yanlış komut yazması, olmayan bir eşya veya NPC ile etkileşime girmeye çalışması gibi durumlarda oyun uygun hata mesajları üretmekte ve oyunun akışı bozulmadan devam etmektedir. Ayrıca, beklenmeyen sistemsel hatalar için oyun döngüsü içerisinde geniş kapsamlı bir **try-catch** yapısı kullanılmıştır. Böylece NullPointerException gibi hatalar oyunu tamamen sonlandırmadan kullanıcıya bilgi vererek sürdürülebilir bir deneyim sunmaktadır.

### 7.1 Uygulanan Hata Kontrolleri ve Mesajları

Oyun içinde çeşitli durumlara özel hata mesajları bulunmaktadır:

#### • Geçersiz komut

Kullanıcı desteklenmeyen bir komut yazdığında:

```
Örnek komutlar: bak | git kuzey | git guney | env | durum | al küçük_iksir | konuş Goblin
Komut bekleniyor (yardım için 'yardim'): uc
Bilinmeyen komut. Yardım için 'yardim' yazabilirsin.
```

- **Odada bulunmayan eşya veya karakter**

Kullanıcı olmayan bir eşya almaya çalıştığında:

```
Örnek komutlar: bak | git kuzey | git guney | env | durum | al kucuk_iksir | konus Goblin  
Komut bekleniyor (yardim için 'yardim'): al mavi_anahtar  
Böyle bir eşya/karakter bulunmuyor.
```

NPC adı yanlış yazıldığında:

```
Örnek komutlar: bak | git kuzey | git guney | env | durum | al kucuk_iksir | konus Goblin  
Komut bekleniyor (yardim için 'yardim'): konus Merve  
Böyle bir eşya/karakter bulunmuyor.
```

- **Envanterde olmayan eşyayı kullanma**

Kullanıcı envanterinde bulunmayan bir eşyayı kullanmaya çalıştığında:

```
Örnek komutlar: bak | git kuzey | git guney | env | durum | al kucuk_iksir | konus Goblin  
Komut bekleniyor (yardim için 'yardim'): kullan demir_kilic  
Bu eşya sende yok.
```

- **Çıkışı olmayan yöne gitme**

Oyuncunun geçiş olmayan bir yöne gitmesi durumunda:

```
Örnek komutlar: bak | git kuzey | git guney | env | durum | al kucuk_iksir | konus Goblin  
Komut bekleniyor (yardim için 'yardim'): git batı  
Bu yönde çıkış yok.
```

## 7.2 Sistemsel İstisna Yönetimi

Oyun döngüsü içinde tüm komutların işlendiği ana bölüm bir **try-catch** bloğu ile korunmuştur:

```
try {  
    komutIsle(giris.trim());  
} catch (Exception e) {  
    System.out.println("Beklenmeyen bir hata oluştu, ama oyun devam ediyor.");  
    System.out.println("Hata mesajı: " + e.getMessage());  
}
```

Bu sayede:

- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException
- vb. hatalar

oyunu tamamen durdurmadan güvenli şekilde yakalanır.

### 7.3 Bu Tasarımın Sağladığı Avantajlar

- Oyun hiçbir durumda çökmekte, kullanıcı deneyimi korunmaktadır.
- Kullanıcı yanlış komutlarda kaybolmamaktadır.
- Hatalar açık ve anlaşılır mesajlarla iletilmektedir.
- Polimorfik ve çok sınıflı yapılara rağmen sistem stabil çalışmaktadır.
- Öğrencilerin OOP mantığını uygularken gerçek bir hata yönetimi pratiği kazanmalarını sağlar.

## 8.Kaynak Kodlar Hakkında Açıklamalar

Projenin tüm kaynak kodları, teslim kolaylığı ve bütünlüğün korunması amacıyla **tek bir .java dosyasında** toplanmıştır. Dosya içerisinde sınıflar mantıksal bir sıraya göre konumlandırılmıştır. Bu yapı, hem inceleme sürecini kolaylaştırmakta hem de sınıfların birbirleriyle olan ilişkilerini net bir biçimde ortaya koymaktadır.

### 8.1 Dosya Yapısının Genel Düzeni

Kaynak dosyada sınıflar şu sırayla yer almaktadır:

#### 1. Main (Ana sınıf)

- Programın giriş noktasıdır.
- Sadece OyunMotoru sınıfını başlatmakla görevlidir.
- Bu sayede *ayrık sorumluluk prensibi* (SRP) korunmuştur.

#### 2. OyunMotoru (Game Engine)

- Tüm oyun akışını, komutların işlenmesini ve dünya düzeninin kurulmasını yönetir.
- Kullanıcıdan komut alma, yorumlama ve uygun sınıflara yönlendirme görevleri buradadır.
- Genel try-catch yapısı ile uygulama düzeyinde istisna yönetimi sağlar.

### 3. Oda (Room) ve Alt Sınıfları

- Oda: Temel oda sınıfıdır. Odalar arası bağlantılar, eşya ve NPC listeleri burada tutulur.
- TuzakOdasi: Kalıtım ile Oda sınıfından türemiştir. Override edilmiş odaGirildi() metodunda oyuncuya zarar veren davranış eklenmiştir.
- IyileşmeOdasi: Aynı şekilde Oda sınıfından türetilmiştir. İlk girişte oyuncuyu iyileştirir.  
→ Bu yapı, **kalıtım + metod geçersiz kılma (override)** kullanımının net bir örneğidir.

### 4. Eşya (Item) Soyut Sınıfı ve Alt Türleri

- Eşya: Soyut (abstract) bir üst sınıftır. Tüm eşyalarda ortak özellikler (id, komut adı, açıklama) burada tanımlanmıştır.
- AnahtarEşya: Kapıları açmak için kullanılır.
- IksirEşya: Oyuncunun canını artırır.
- SilahEşya: Oyuncunun saldırı gücünü artırır.  
→ Bu yapı, **polimorfizmin uygulandığı alanlardan biridir.**  
→ Oyuncu bir eşya kullandığında hangi sınıfa ait olduğunu bilmek zorunda değildir, davranış tamamen alt sınıflar tarafından belirlenir.

### 5. Oyuncu (Player)

- Oyuncunun durumunu (can, saldırı gücü, envanter, bulunduğu oda) yönetir.
- Hareket, eşya alma ve eşya kullanma gibi işlemler burada gerçekleştirilir.

### 6. NPC Hiyerarşisi (Karakterler)

- NPC: Tüm karakterler için temel sınıftır.
- DostNPC: Diyalog kurabilen karakterleri temsil eder.
- DusmanNPC: Oyuncuya saldıran karakterleri temsil eder.  
→ Bu ayrım, kalıtım sayesinde davranış çeşitliliği sağlar.

### 7. Konuşma Yapısı

- KonuşmaDugumu: Diyalog metinlerini tutar.

- KonuşmaSecenegi: Diyalog seçeneklerini ve bu seçeneğin bağlı olduğu diğer düğümü temsil eder.  
→ Böylece **ağaç şeklinde ilerleyebilen dinamik bir konuşma sistemi** oluşturulmuştur.

## 8.2 Kodun Genel Yazım Kalitesi

Proje kodunda:

- Mantıksal sınıf ayrımı
- Fonksiyonların tek bir sorumluluğu olması
- Türkçe değişken ve metot adlandırması
- Okunabilirlik için boşluk ve format düzeni
- Gereksiz tekrarların azaltılması

gibi yazılım geliştirme prensiplerine dikkat edilmiştir.

Bu nedenle kaynak kodlar hem anlaşılır hem de ders kapsamında örnek gösterilmeye uygun bir yapıdadır.

## 9.Test Senaryoları

Aşağıdaki test senaryoları, geliştirilen metin tabanlı mini macera oyununun temel işlevlerinin doğru çalışıp çalışmadığını doğrulamak amacıyla hazırlanmıştır. Her senaryoda kullanıcı tarafından verilen komut ve oyundan beklenen çıktı belirtilmiştir.

| No | Test Senaryosu (Komut) | Beklenen Sonuç                                                                                                           |
|----|------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 1  | bak                    | Oyuncunun bulunduğu odanın adı, açıklaması, mevcut çıkışlar, odadaki eşyalar ve NPC'ler doğru şekilde yazdırılır.        |
| 2  | al kirmizi_anahtar     | Kırmızı anahtar envantere eklenir ve bulunduğu odadan kaldırılır. "kirmizi_anahtar envanterine eklendi." mesajı görünür. |
| 3  | kullan kirmizi_anahtar | Salonun doğu yönündeki kırmızı kapı açılır, kilit kalkar ve oyuncu Silah Odası'na geçer. Odanın detayları ekrana         |

|    |                              |                                                                                                                                                |
|----|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
|    |                              | yazdırılır.                                                                                                                                    |
| 4  | konus Muhafız                | Dost NPC olan Muhafız'ın diyalog metni ve konuşma seçenekleri ekrana yazdırılır. Ardından kullanıcıdan sec <numara> girişi beklenir.           |
| 5  | zort komutu (geçersiz komut) | “Bilinmeyen komut. Yardım için 'yardim' yazabilirsin.” uyarısı gösterilir. Oyun kapanmaz.                                                      |
| 6  | kullan kucuk_iksir           | İksir kullanıldığında oyuncunun canı artar. “kucuk_iksir içtin. Canın X arttı.” mesajı görüntülenir.                                           |
| 7  | kullan demir_kilic           | Silah kuşanılır, saldırı gücü artar. “demir_kilic kuşandın. Saldırı gücün +5 arttı.” mesajı gösterilir.                                        |
| 8  | git kuzey                    | Eğer kuzeyde bir çıkış varsa oyuncu ilgili odaya geçer. Yoksa “Bu yönde çıkış yok.” mesajı görülür.                                            |
| 9  | git dogu (kapı kilitliyen)   | “Kırmızı kapı kilitli.” uyarısı gösterilir ve oyuncu odadan çıkamaz.                                                                           |
| 10 | kullan olmayan_esya          | Oyuncunun envanterinde bulunmayan bir eşya kullanıldığında “Bu eşya sende yok.” uyarısı verilir.                                               |
| 11 | konus Goblin                 | Goblin dost NPC olmadığı için konuşmak yerine saldırır. Oyuncu hasar alır. Eğer Goblin'in canı sıfıra düşerse odadan kaldırılır.               |
| 12 | Tuzak Odasına giriş          | Oyuncu odanın ilk girişinde tuzak tetiklenir ve oyuncu hasar alır. Sonraki girişlerde tuzak çalışmaz.                                          |
| 13 | İyileşme Odasına giriş       | Oyuncu odanın ilk girişinde 30 can iyileşir. Sonraki girişlerde iyileştirme tekrar etmez.                                                      |
| 14 | sec 1 (Konuşma içinde)       | Kullanıcı geçerli bir seçenek numarası girerse konuşma akışı doğru şekilde ilerler. Geçersizse “Geçersiz seçim numarası.” mesajı görüntülenir. |

## 10. Sonuç ve Değerlendirme

Bu proje kapsamında geliştirdiğim metin tabanlı mini macera oyunu, Java ile nesne tabanlı programlama (OOP) ilkelerini gerçek bir senaryo üzerinde uygulamalı olarak deneyimlememi sağlamıştır. Oyunda; odalar arası geçiş yapma, eşya toplama ve kullanma, dost ve düşman NPC'lerle etkileşime girme, tuzak odaları ve iyileştirme odaları gibi dinamik oyun mekaniklerinin tamamı nesne yönelimli bir yapı ile bir araya getirilmiştir.

Oluşturduğum sınıf yapısı, özellikle **kalıtım (inheritance)**, **çok biçimlilik (polimorfizm)** ve **kompozisyon** kavramlarının oyun dünyasına nasıl entegre edilebileceğini açık bir şekilde göstermektedir. Örneğin; Oda → TuzakOdasi/IyilemeOdasi, Eşya → AnahtarEşya/IksirEşya/SilahEşya ve NPC → DostNPC/DusmanNPC hiyerarşileri üzerinden farklı davranışlar üretmek, proje sürecinde OOP mantığını daha iyi kavramama katkı sağlamıştır. Aynı şekilde konuşma yapısının bir ağaç modeli şeklinde tasarlanması, OOP'nin soyutlama ve modülerlik avantajlarını pratikte görmeme olanak tanımıştır.

Kullanıcı etkileşimi tamamen komut bazlı olduğu için oyun, gerçek zamanlı bir konsol akışı üzerinde çalışmaktadır. Bu süreçte hatalı kullanıcı girişlerinin yönetimi, try-catch bloklarının doğru kullanımı ve istisnalar oluştuğunda programın kesilmeden devam etmesi gibi konularda da tecrübe kazandım. Bu özellikler, projenin hem kullanıcı dostu hem de dayanıklı bir yapıda olmasına katkı sağlamıştır.

Genel olarak proje, hem yazılım tasarımına dair teori kısmını hem de pratik uygulamaları bir araya getirerek öğretici ve geliştirici bir çalışma olmuştur. Ayrıca kodun modüler ve genişletilebilir olması sayesinde ileride oyuna:

- daha fazla oda veya harita bölgesi,
- yeni eşya türleri,
- gelişmiş savaş mekanikleri,
- görev (quest) sistemi,
- kayıt/yükleme (save/load) desteği,
- daha karmaşık NPC yapay zekâsı

gibi özellikler kolaylıkla eklenebilecektir.

Sonuç olarak proje, nesne tabanlı programlamayı anlamam, sınıf yapıları arasında ilişkiler kurmam, kalıtım ve polimorfizmi etkin bir şekilde kullanmam ve çalışır durumda bir oyun geliştirmem açısından oldukça faydalı ve tamamlayıcı bir deneyim olmuştur.



## GENEL DEĞERLENDİRME TABLOSU

Aşağıdaki tablo, geliştirilen metin tabanlı macera oyununun OOP ilkelerine uygunluğu, teknik yeterliliği, oyun akışı, kullanıcı deneyimi ve hata yönetimi açısından genel değerlendirmesini özetlemektedir.

| Değerlendirme Kriteri          | Açıklama                                                                                                                           | Puan (1-5) |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>OOP Yapısına Uygunluk</b>   | Sınıfların düzenli ayrılması, kalıtım hiyerarşisi (Oda, Eşya, NPC), polimorfizm ve soyut sınıfların başarılı kullanımı.            | 5          |
| <b>Kalıtım Kullanımı</b>       | Oda → TuzakOdasi / İyileşmeOdasi, Eşya → AnahtarEşya / IksirEşya / SilahEşya, NPC → DostNPC / DusmanNPC yapıları doğru uygulanmış. | 5          |
| <b>Polimorfizm Kullanımı</b>   | Eşya sınıfında kullan() metodunun alt sınıflar tarafından farklı davranış sergilemesi; Oda'nın override edilmesi.                  | 5          |
| <b>Kullanıcı Etkileşimi</b>    | Türkçe komut sistemi, dinamik komut önerileri, net hata mesajları ve akıcı oyun akışı.                                             | 4          |
| <b>Hata Yönetimi</b>           | Geçersiz komutlar, olmayan eşya, kilitli kapı ve sistemsel hataların try-catch ile kontrol edilmesi.                               | 5          |
| <b>Kod Okunabilirliği</b>      | Yorum satırları, düzenli sınıf yapısı, anlaşılır değişken adları.                                                                  | 5          |
| <b>Oyun İçerik Çeşitliliği</b> | 5+ oda, tuzak/iyileşme mekanikleri, diyalog sistemi, düşman saldırısı, envanter sistemi.                                           | 4          |
| <b>Genişletilebilirlik</b>     | Yeni oda, eşya veya NPC eklemeye uygun mimari; modüler yapı.                                                                       | 5          |
| <b>Performans ve Stabilité</b> | Konsol tabanlı uygulama için yeterli performans; döngüde beklenmeyen kapanma yok.                                                  | 5          |